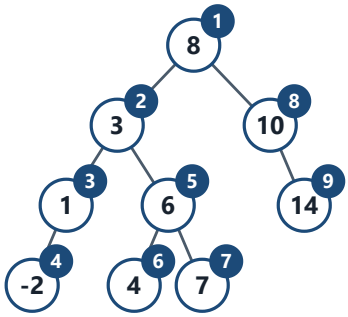
Ôn tập DSA: Sắp xếp · Tìm kiếm · Cây

Bản đầy đủ có giải thích, theo cách làm của cô Hồng. Dành cho bài kiểm tra: mỗi câu gồm 3 ý: a) duyệt cây, b) khai báo, c) viết hàm.

Cách dùng: đọc phần A (cây) trước vì đề mẫu Câu 8 nằm ở đó, rồi B (cây tìm kiếm), C (sắp xếp), D (tìm kiếm). Mỗi phần có **code**, **hình** và **ví dụ có đáp án**. Cuối tài liệu là phụ lục các bước sắp xếp cho 3 dãy số trong đề của cô. Khung xanh là đáp án; khung vàng là lỗi hay mất điểm.



Đề mẫu Câu 8, thứ tự trước: 8 3 1 -2 6 4 7 10 14

Phần	Nội dung	Dạng đề
A	Cây nhị phân: khai báo, duyệt, hàm đệ quy	Câu 8 a, b, c; 6 bài đệ quy
B	Cây nhị phân tìm kiếm: dựng, bổ sung, hủy nút	Bài 1, Bài 2 (dựng và hủy)
C	Sắp xếp: chọn, chèn, nổi bọt, trộn, nhanh	Viết hàm tăng/giảm dần
D	Tìm kiếm: tuần tự, nhị phân	Ví dụ 5 (x = 20)
E	Phụ lục: các bước 5 thuật toán trên 3 dãy số	Trình bày các bước

A. Cây nhị phân

A1. Khái niệm và khai báo

Cây nhị phân: mỗi nút có **tối đa hai con**, phân biệt **con trái** và **con phải**. **Gốc** là nút trên cùng; **lá** là nút không có con; **mức** của gốc là 0; **chiều cao** là số nút trên đường dài nhất từ gốc xuống lá (cây rỗng cao 0, cây chỉ có gốc cao 1).

Với cây Câu 8: gốc 8; lá là −2, 4, 7, 14; nút 3 có hai con (1 và 6), nút 1 chỉ có con trái (−2), nút 10 chỉ có con phải (14); chiều cao 4.

Lưu trữ móc nối

Mỗi nút gồm dữ liệu và hai con trỏ. Cây được biểu diễn bằng **con trỏ tới nút gốc**; con trỏ bằng `NULL` là cây rỗng.

```
struct nutCay
{
    int duLieu;
    nutCay *trai;
    nutCay *phai;
};
```

```
nutCay *taoNut(int x)
{
    nutCay *p = new nutCay;
    p->duLieu = x;
    p->trai = p->phai = NULL;
    return p;
}
```

Chú ý khi viết b): khai báo **đủ ba trường** (`duLieu` , `trai` , `phai`), con trỏ kiểu `nutCay *` , và **dấu `;` sau dấu `}`** . Đề nói "mỗi nút lưu một số nguyên" nên `duLieu` kiểu `int` .

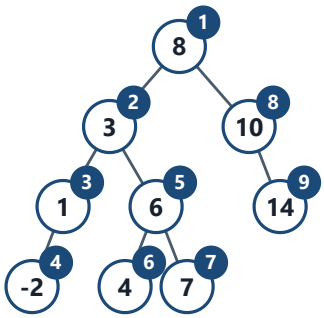
Lưu trữ bằng mảng (cây đầy đủ, đánh số từ 0)

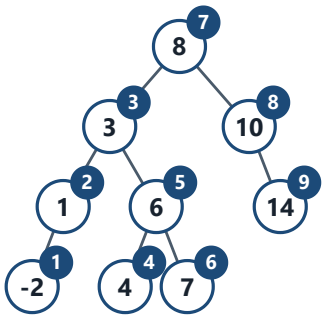
Nút i có con trái ở `2*i + 1`, con phải ở `2*i + 2`; cha của nút j ở `(j - 1) / 2`. Ví dụ nút 4 có con 9 và 10; nút 9 có cha là 4.

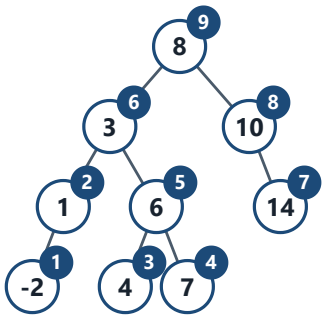
A2. Ba kiểu duyệt cây

Ba hàm giống hệt nhau, **chỉ khác vị trí dòng** `cout`: in gốc *trước* hai cây con (trước), *giữa* hai cây con (giữa), hay *sau* hai cây con (sau). Cách nhớ: "trước/giữa/sau" là vị trí của **gốc** so với trái và phải.

```
void duyetTruoc(nutCay *t)      // gốc, trái, phải
{
    if (t != NULL)
    {
        cout << t->duLieu << " ";
        duyetTruoc(t->trai);
        duyetTruoc(t->phai);
    }
}
void duyetGiua(nutCay *t)       // trái, gốc, phải   (BST: tăng dần)
{
    if (t != NULL)
    {
        duyetGiua(t->trai);
        cout << t->duLieu << " ";
        duyetGiua(t->phai);
    }
}
void duyetSau(nutCay *t)        // trái, phải, gốc
{
    if (t != NULL)
    {
        duyetSau(t->trai);
        duyetSau(t->phai);
        cout << t->duLieu << " ";
    }
}
```



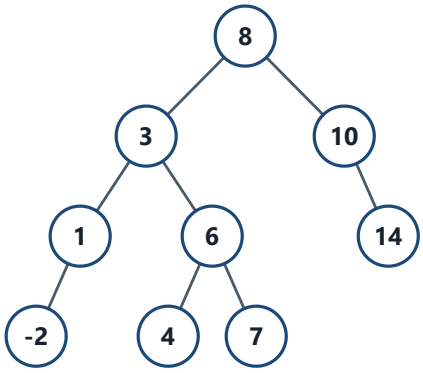




Trước (gốc, trái, phải): 8 3 1 -2 6 4 7 10 14 **Giữa (trái, gốc, phải)**: -2 1 3 4 6 7 8 10 14 **Sau (trái, phải, gốc)**: -2 1 4 7 6 3 14 10 8

Đọc nhanh: trước = `8 3 1 -2 6 4 7 10 14` · giữa = `-2 1 3 4 6 7 8 10 14` · sau = `-2 1 4 7 6 3 14 10 8`.
Số trong chấm xanh là thứ tự nút được thăm. Cây tìm kiếm nhị phân thì duyệt giữa cho dãy tăng dần (đúng với Câu 8: −2 1 3 4 6 7 8 10 14).

A3. Đề mẫu Câu 8 làm trọn



Hình minh họa cây nhị phân cho câu a

a) Thứ tự duyệt. Cách làm từng bước (duyệt trước): thăm 8; sang trái thăm 3; sang trái thăm 1; sang trái thăm −2 (hết nhánh), lùi lại; 1 không có con phải, lùi về 3; sang phải thăm 6, rồi 4, rồi 7; lùi về 8; sang phải thăm 10; 10 không có con trái; sang phải thăm 14. Kết quả:

Trước: `8 3 1 -2 6 4 7 10 14`
Giữa: `-2 1 3 4 6 7 8 10 14`
Sau: `-2 1 4 7 6 3 14 10 8`

b) Khai báo (lưu trữ móc nối, mỗi nút một số nguyên):

```
struct nutCay
{
    int duLieu;
    nutCay *trai;
    nutCay *phai;
};
```

c) Hàm tính tổng các nút có giá trị dương. Nút dương thì lấy `duLieu` cộng với kết quả hai cây con; nút không dương thì chỉ gộp hai cây con.

```
long long tongDuong(nutCay *t)
{
    if (t == NULL) return 0;               // cây rỗng: dừng
    if (t->duLieu > 0)                     // nút dương: cộng giá trị của nó
        return t->duLieu + tongDuong(t->trai) + tongDuong(t->phai);
    return tongDuong(t->trai) + tongDuong(t->phai);   // nút âm hoặc 0: bỏ qua
}
```

Nút	8	3	1	-2	6	4	7	10	14	Tổng
Đóng góp	8	3	1	0	6	4	7	10	14	**53**

Lỗi hay gặp: viết `return (t->duLieu > 0) + ...` chỉ *đếm* số nút dương (được 8), không *cộng giá trị* (phải được 53). Nút −2 phải bị bỏ qua, không trừ.

A4. Khung đệ quy trên cây (3 bước)

Mọi hàm "đếm / tính tổng trên cây" đều theo một khung:

1. **Cây rỗng** (`t == NULL`): trả về 0, đây là điều kiện dừng.
2. **Phần đóng góp của riêng nút t**.
3. **Gộp:** phần của nút + kết quả cây trái + kết quả cây phải.

```
int demNut(nutCay *t)
{
    if (t == NULL) return 0;
    return 1 + demNut(t->trai) + demNut(t->phai);
}
int demLe(nutCay *t)
{
    if (t == NULL) return 0;
    if (t->duLieu % 2 != 0)         // không viết == 1: -15 % 2 bằng -1
        return 1 + demLe(t->trai) + demLe(t->phai);
    return demLe(t->trai) + demLe(t->phai);
}
long long tongNut(nutCay *t)
{
    if (t == NULL) return 0;
    return t->duLieu + tongNut(t->trai) + tongNut(t->phai);
}
long long tongDuong(nutCay *t)
{
    if (t == NULL) return 0;
    if (t->duLieu > 0)
        return t->duLieu + tongDuong(t->trai) + tongDuong(t->phai);
    return tongDuong(t->trai) + tongDuong(t->phai);
}
```

```
int demLa(nutCay *t)
{
    if (t == NULL) return 0;
    if (t->trai == NULL && t->phai == NULL) return 1;
    return demLa(t->trai) + demLa(t->phai);
}
int chieuCao(nutCay *t)             // cây rỗng cao 0, chỉ có gốc cao 1
{
    if (t == NULL) return 0;
    int hTrai = chieuCao(t->trai);
    int hPhai = chieuCao(t->phai);
    return 1 + (hTrai > hPhai ? hTrai : hPhai);
}
nutCay *timKiem(nutCay *t, int x)   // BST: NULL nếu không có
{
    if (t == NULL) return NULL;
    if (x == t->duLieu) return t;
    if (x < t->duLieu) return timKiem(t->trai, x);
    return timKiem(t->phai, x);
}
nutCay *nhoNhat(nutCay *t)          // đi mãi sang trái
{
    if (t == NULL) return NULL;
    while (t->trai != NULL) t = t->trai;
    return t;
}
```

Kết quả trên cây Câu 8

Hàm	Kết quả	Giải thích
`demNut`	9	8, 3, 1, −2, 6, 4, 7, 10, 14
`demLa`	4	−2, 4, 7, 14
`chieuCao`	4	đường 8 → 3 → 1 → −2 có 4 nút
`demLe`	3	3, 1, 7 (−2 là chẵn)
`tongNut`	51	8+3+1−2+6+4+7+10+14
`tongDuong`	53	bỏ −2: 51 + 2

Số lẻ âm: `-15 % 2` bằng `-1` trong C++, nên viết `% 2 != 0`, không viết `== 1`.

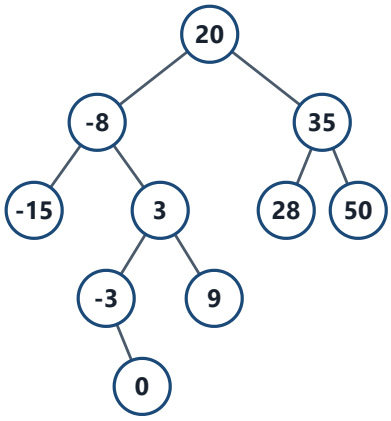
A5. Sáu bài tập đệ quy của cô (de_quy.txt), có lời giải

1. Khai báo cây nhị phân lưu trữ móc nối, mỗi nút một số nguyên → `struct nutCay` (A1).
2. Thủ tục bổ sung x vào BST có gốc trỏ bởi T → `boSung` (B3).
3. Đếm số nút → `demNut`.
4. Đếm số nút lẻ → `demLe`.
5. Tổng giá trị các nút → `tongNut`.
6. Tổng giá trị các nút dương → `tongDuong` (A3, A4).

Chỉ khác nhau ở **phần đóng góp của nút**: đếm nút là `1`; đếm lẻ là `(t->duLieu % 2 != 0)`; tổng là `t->duLieu`; tổng dương là `(t->duLieu > 0 ? t->duLieu : 0)`. Học khung một lần, thay phần này là xong.

A6. Hai đề luyện (tự làm rồi đối chiếu)

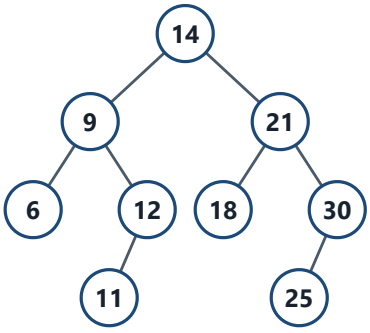
Đề 1. Dựng BST từ dãy `20 -8 35 -15 3 28 50 -3 9 0`. a) Duyệt trước, giữa, sau. b) Đếm nút lẻ. c) Tổng nút dương.



BST dựng từ dãy của Đề 1

Trước: `20 -8 -15 3 -3 0 9 35 28 50`
Giữa: `-15 -8 -3 0 3 9 20 28 35 50` (tăng dần)
Sau: `-15 0 -3 9 3 -8 28 50 35 20`
Nút lẻ: **5** (-15, -3, 3, 9, 35) · Tổng nút dương: **145** · Tổng tất cả: 119

Đề 2. Dựng BST từ dãy `14 9 21 6 12 18 30 11 25`. a) Duyệt trước, giữa, sau. b) Đếm nút chia hết cho 3. c) Chiều cao cây.



BST dựng từ dãy của Đề 2

Trước: `14 9 6 12 11 21 18 30 25`
Giữa: `6 9 11 12 14 18 21 25 30`
Sau: `6 11 12 9 18 25 30 21 14`
Chia hết cho 3: **6** (6, 9, 12, 18, 21, 30) · Chiều cao: **4**

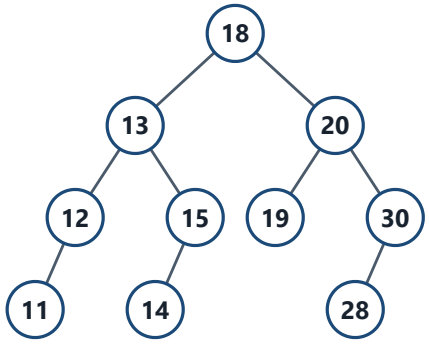
B. Cây nhị phân tìm kiếm (BST)

B1. Định nghĩa và dựng cây

BST là cây nhị phân trong đó **mọi nút** có khóa lớn hơn tất cả khóa ở cây con trái và nhỏ hơn tất cả khóa ở cây con phải; các khóa khác nhau. Dựng cây: bắt đầu từ cây rỗng, thêm lần lượt từng khóa. Mỗi khóa đi từ gốc: **nhỏ hơn thì sang trái, lớn hơn thì sang phải**, gặp chỗ trống thì đặt nút mới ở đó. Khóa đã có thì bỏ qua.

Bài 1a: dãy 18 13 15 12 20 11 30 19 28 14

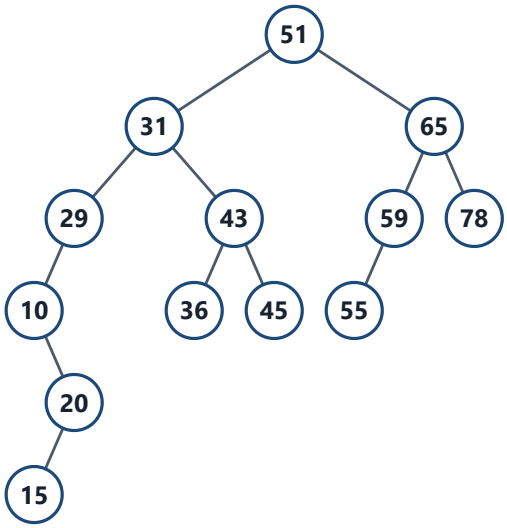
Khóa	Đường đi từ gốc
18	Cây rỗng, nên 18 là **gốc**
13	13 < 18, sang trái → đặt làm con **trái** của 18
15	15 < 18, sang trái; 15 > 13, sang phải → đặt làm con **phải** của 13
12	12 < 18, sang trái; 12 < 13, sang trái → đặt làm con **trái** của 13
20	20 > 18, sang phải → đặt làm con **phải** của 18
11	11 < 18, sang trái; 11 < 13, sang trái; 11 < 12, sang trái → đặt làm con **trái** của 12
30	30 > 18, sang phải; 30 > 20, sang phải → đặt làm con **phải** của 20
19	19 > 18, sang phải; 19 < 20, sang trái → đặt làm con **trái** của 20
28	28 > 18, sang phải; 28 > 20, sang phải; 28 < 30, sang trái → đặt làm con **trái** của 30
14	14 < 18, sang trái; 14 > 13, sang phải; 14 < 15, sang trái → đặt làm con **trái** của 15



Cây BST của Bài 1a

Bài 2a: dãy 51 31 43 29 65 10 20 36 78 59 15 45 55

Khóa	Đường đi từ gốc
51	Cây rỗng, nên 51 là **gốc**
31	31 < 51, sang trái → đặt làm con **trái** của 51
43	43 < 51, sang trái; 43 > 31, sang phải → đặt làm con **phải** của 31
29	29 < 51, sang trái; 29 < 31, sang trái → đặt làm con **trái** của 31
65	65 > 51, sang phải → đặt làm con **phải** của 51
10	10 < 51, sang trái; 10 < 31, sang trái; 10 < 29, sang trái → đặt làm con **trái** của 29
20	20 < 51, sang trái; 20 < 31, sang trái; 20 < 29, sang trái; 20 > 10, sang phải → đặt làm con **phải** của 10
36	36 < 51, sang trái; 36 > 31, sang phải; 36 < 43, sang trái → đặt làm con **trái** của 43
78	78 > 51, sang phải; 78 > 65, sang phải → đặt làm con **phải** của 65
59	59 > 51, sang phải; 59 < 65, sang trái → đặt làm con **trái** của 65
15	15 < 51, sang trái; 15 < 31, sang trái; 15 < 29, sang trái; 15 > 10, sang phải; 15 < 20, sang trái → đặt làm con **trái** của 20
45	45 < 51, sang trái; 45 > 31, sang phải; 45 > 43, sang phải → đặt làm con **phải** của 43
55	55 > 51, sang phải; 55 < 65, sang trái; 55 < 59, sang trái → đặt làm con **trái** của 59



Cây BST của Bài 2a

Kiểm tra bằng duyệt giữa phải tăng dần. Bài 1a: `11 12 13 14 15 18 19 20 28 30` . Bài 2a: `10 15 20 29 31 36 43 45 51 55 59 65 78` .

B2. Bổ sung, tìm kiếm, nhỏ nhất

Tham số `nutCay *&t` là **tên khác của chính ô con trỏ** được truyền vào, nên `t = taoNut(x)` ghi thẳng vào cây thật. Nếu quên dấu `&`, cây không thay đổi.

```
void boSung(nutCay *&t, int x)      // t là tham chiếu: sửa thẳng vào cây thật
{
    if (t == NULL)
        t = taoNut(x);
    else if (x < t->duLieu)
        boSung(t->trai, x);
    else if (x > t->duLieu)
        boSung(t->phai, x);
    // x == t->duLieu: đã có, không làm gì
}
```

```
nutCay *timKiem(nutCay *t, int x)   // BST: NULL nếu không có
{
    if (t == NULL) return NULL;
    if (x == t->duLieu) return t;
    if (x < t->duLieu) return timKiem(t->trai, x);
    return timKiem(t->phai, x);
}
nutCay *nhoNhat(nutCay *t)          // đi mãi sang trái
{
    if (t == NULL) return NULL;
    while (t->trai != NULL) t = t->trai;
    return t;
}
```

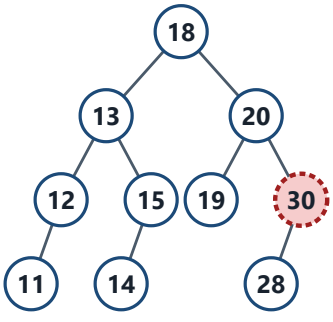
B3. Hủy nút

Tìm nút x như khi tìm kiếm, rồi xét số con:

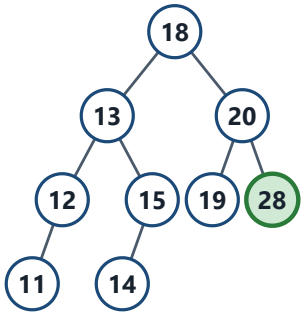
1. **Lá:** bỏ nút, cha của nó trỏ `NULL`.
2. **Một con:** cha của nút trỏ thẳng tới con duy nhất của nó.
3. **Hai con:** chọn **nút thế mạng**, chép khóa của nó lên nút cần hủy, rồi hủy nút thế mạng cũ (nút này có tối đa một con).
   · **Cách hai (theo sách):** nút thế mạng là **trái nhất của cây con phải**.
   · **Cách một:** nút thế mạng là **phải nhất của cây con trái**.

```
void timTheMang(nutCay *&p, nutCay *&q)   // cách hai: trái nhất của cây con phải
{
    if (q->trai != NULL)
        timTheMang(p, q->trai);
    else
    {
        p->duLieu = q->duLieu;   // chép khóa của q lên nút cần hủy
        p = q;                   // p trỏ tới nút sẽ bị hủy thật sự
        q = q->phai;             // nối cha của q với con phải của q
    }
}
int xoaNut(nutCay *&t, int x)             // 1: hủy được, 0: không có x
{
    if (t == NULL) return 0;
    if (x < t->duLieu) return xoaNut(t->trai, x);
    if (x > t->duLieu) return xoaNut(t->phai, x);
    nutCay *p = t;               // t->duLieu == x
    if (t->trai == NULL)
        t = t->phai;             // lá, hoặc chỉ có con phải
    else if (t->phai == NULL)
        t = t->trai;             // chỉ có con trái
    else
        timTheMang(p, t->phai);  // đủ hai con
    delete p;
    return 1;
}
```

Bài 1b: hủy 30, rồi 20

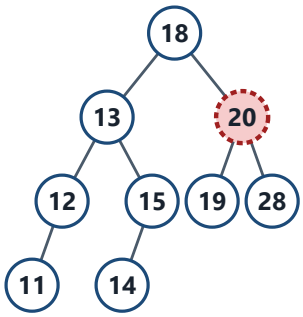


Trước: nút 30 (đỏ)

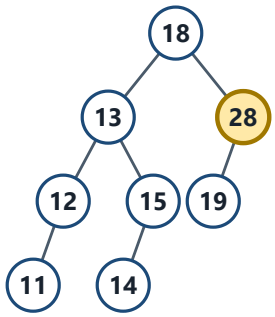


Sau hủy 30 (một con): 28 nối lên

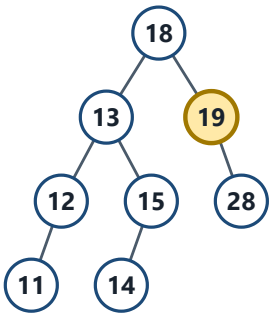
Hủy 30: nút 30 chỉ có **một con** (con trái là 28). Cha của 30 là 20, nên cho con phải của 20 trỏ thẳng tới 28 rồi giải phóng 30.



Trước khi hủy 20 (đỏ), 20 có hai con 19 và 28



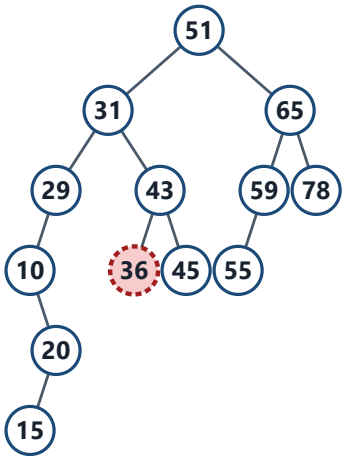
Cách hai: 28 thế mạng



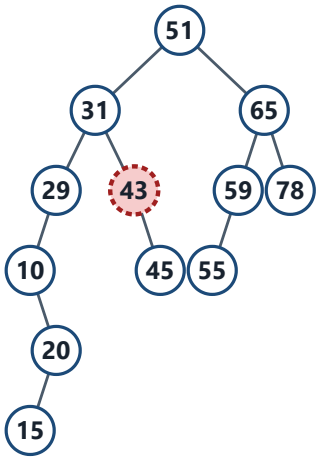
Cách một: 19 thế mạng

Hủy 20 (hai con): cách hai, cây con phải của 20 chỉ có 28 (không có con trái), nên 28 là nút trái nhất: chép 28 lên nút 20, hủy nút 28 cũ. Cách một, nút phải nhất của cây con trái (19) là 19: chép 19 lên nút 20, hủy nút 19 cũ. Cả hai đều cho BST hợp lệ.

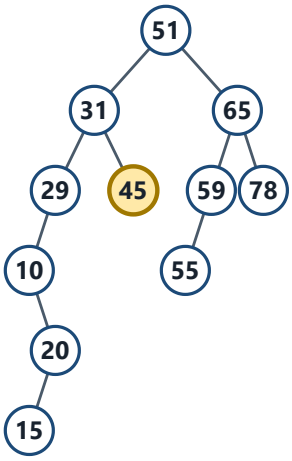
Bài 2b: hủy 36, rồi 43



Trước: hủy 36 (đỏ)



Sau hủy 36 (lá). Tiếp: hủy 43 (đỏ)



Sau hủy 43 (một con): 45 nối lên

Hủy 36: nút lá (con trái của 43), cho con trái của 43 bằng `NULL` rồi giải phóng. **Hủy 43:** lúc này chỉ còn con phải 45, cho con phải của 31 trỏ thẳng tới 45 rồi giải phóng 43.

C. Sắp xếp

Đề yêu cầu: (a) mô tả và viết hàm cho 5 thuật toán, mỗi cái **tăng dần** và **giảm dần**; (b) trình bày các bước trên 3 dãy số. Code dưới đây là sắp **tăng dần**; phần tô vàng là chỗ **đảo dấu** để được bản giảm dần. Tất cả mảng đánh số từ 0.

Thuật toán	Ý tưởng	Tốt nhất	Xấu nhất	Ổn định	Mảng phụ
Chọn	Chọn phần tử nhỏ nhất của phần chưa sắp, đổi về đầu	n^2	n^2	không	không
Chèn	Chèn từng phần tử vào đúng chỗ trong phần đầu đã sắp	n	n^2	có	không
Nổi bọt	So cặp kề, sai thứ tự thì đổi; phần tử lớn nhất nổi dần về cuối	n^2 (có cờ: n)	n^2	có	không
Trộn	Chia đôi, sắp từng nửa, trộn hai nửa đã sắp	n log n	n log n	có	O(n)
Nhanh	Chọn pivot, đưa nhỏ hơn sang trái, lớn hơn sang phải, sắp hai phía	n log n	n^2	không	không

C1. Sắp xếp chọn (Selection Sort)

Lượt i: tìm phần tử nhỏ nhất trong `a[i..n-1]`, đổi chỗ với `a[i]`. Sau lượt i, đầu mảng đã đúng vị trí. Giảm dần: tìm phần tử **lớn nhất**.

```
void SelectionSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int vt = i;                          // vị trí phần tử nhỏ nhất
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[vt])
                vt = j;
        swap(a[i], a[vt]);
    }
}
```

C2. Sắp xếp chèn (Insertion Sort)

Phần `a[0..i-1]` đã sắp. Lấy `x = a[i]`, dồn các phần tử lớn hơn `x` sang phải một ô, đặt `x` vào chỗ trống. Giống xếp bài trên tay.

```
void InsertionSort(int a[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int x = a[i], j = i - 1;
        while (j >= 0 && a[j] > x)        // dồn các phần tử lớn hơn x sang phải
        {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = x;                      // chèn x vào chỗ trống
    }
}
```

C3. Sắp xếp nổi bọt (Bubble Sort)

Mỗi lượt duyệt cặp kề, đổi chỗ nếu sai thứ tự. Sau lượt i, i + 1 phần tử lớn nhất đã về cuối, nên vòng trong chỉ chạy tới `n - 1 - i`.

```
void BubbleSort(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1])
                swap(a[j], a[j + 1]);
}
```

C4. Sắp xếp trộn (Merge Sort)

Chia đôi tại `giua = (dau + cuoi) / 2`, sắp đệ quy từng nửa, rồi `Merge` hai nửa đã sắp bằng mảng phụ `b`. Dùng `<=` khi so sánh để bằng nhau thì lấy phần tử bên trái trước (ổn định).

```
void Merge(int a[], int dau, int giua, int cuoi)
{
    int b[1000], i = dau, j = giua + 1, k = 0;
    while (i <= giua && j <= cuoi)
        if (a[i] <= a[j]) b[k++] = a[i++];
        else b[k++] = a[j++];
    while (i <= giua) b[k++] = a[i++];
    while (j <= cuoi) b[k++] = a[j++];
    for (i = dau, k = 0; i <= cuoi; i++) a[i] = b[k++];
}
void MergeSort(int a[], int dau, int cuoi)
{
    if (dau < cuoi)
    {
        int giua = (dau + cuoi) / 2;
        MergeSort(a, dau, giua);
        MergeSort(a, giua + 1, cuoi);
        Merge(a, dau, giua, cuoi);
    }
}
```

C5. Sắp xếp nhanh (Quick Sort)

Chọn pivot `x` là phần tử giữa. Con trỏ `i` tiến từ trái tới khi gặp phần tử **không nhỏ hơn** x; `j` lùi từ phải tới khi gặp phần tử **không lớn hơn** x; nếu `i <= j` thì đổi chỗ rồi cả hai tiến vào. Khi hai con trỏ vượt nhau, mảng chia thành `[dau..j]` và `[i..cuoi]`, sắp đệ quy từng phần.

```
void QuickSort(int a[], int dau, int cuoi)
{
    int i = dau, j = cuoi, x = a[(dau + cuoi) / 2];   // pivot = phần tử giữa
    while (i <= j)
    {
        while (a[i] < x) i++;
        while (a[j] > x) j--;
        if (i <= j)
        {
            swap(a[i], a[j]);
            i++;
            j--;
        }
    }
    if (dau < j) QuickSort(a, dau, j);
    if (i < cuoi) QuickSort(a, i, cuoi);
}
```

Giảm dần: chỉ đảo dấu so sánh ở chỗ tô vàng: Chọn `a[j] > a[vt]` (tìm max); Chèn `a[j] < x`; Nổi bọt `a[j] < a[j+1]`; Trộn `a[i] >= a[j]`; Nhanh `a[i] > x` và `a[j] < x`. Không đảo cả vòng lặp.

D. Tìm kiếm

D1. Tìm tuần tự và tìm nhị phân

```
int timTuanTu(int a[], int n, int x)
{
    for (int i = 0; i < n; i++)
        if (a[i] == x) return i;
    return -1;
}
int timNhiPhan(int a[], int n, int x)       // a đã tăng dần
{
    int dau = 0, cuoi = n - 1;
    while (dau <= cuoi)
    {
        int giua = (dau + cuoi) / 2;
        if (a[giua] == x) return giua;
        if (a[giua] < x) dau = giua + 1;
        else cuoi = giua - 1;
    }
    return -1;
}
```

- **Tuần tự:** duyệt từ đầu, gặp x thì trả vị trí, hết mảng thì trả −1. Mảng bất kỳ, O(n).
- **Nhị phân:** chỉ dùng khi mảng **đã sắp tăng dần**. Mỗi lượt so x với phần tử giữa rồi loại một nửa. O(log n).

Điều kiện bắt buộc: viết rõ "mảng đã sắp tăng dần" khi dùng tìm nhị phân. Cô dùng vị trí đếm từ 1 trong ví dụ; code C++ đếm từ 0, nên kết quả vị trí 6 tương ứng chỉ số 5.

D2. Ví dụ 5 của cô: x = 20

A = -8 -5 0 4 15 20 50 63 75 98, vị trí đếm từ 1. Ô vàng là "trung vị" của lượt, ô xám nhạt đã bị loại.

Lượt 1 [1..10]	-8	-5	0	4	**15**	20	50	63	75	98
Lượt 2 [6..10]	-8	-5	0	4	15	20	50	**63**	75	98
Lượt 3 [6..7]	-8	-5	0	4	15	**20**	50	63	75	98

A[5] = 15 < 20 → bỏ nửa trái, [6..10]

A[8] = 63 > 20 → bỏ nửa phải, [6..7]

A[6] = 20 = x → **tìm thấy**

Lượt 1: trung vị là vị trí 5, A[5] = 15 < 20 nên mọi phần tử vị trí 1 đến 5 đều nhỏ hơn x, loại, còn [6..10].
Lượt 2: A[8] = 63 > 20 nên vị trí 9, 10 lớn hơn x, loại, còn [6..7].
Lượt 3: [6..7] có hai phần tử (số chẵn) nên có hai trung vị, chọn trung vị đầu tiên, vị trí 6. A[6] = 20 = x. **Kết luận: x có trong mảng.**

E. Phụ lục: các bước sắp tăng dần cho 3 dãy của cô

Mỗi dòng là mảng **sau** mỗi lượt. Ô xám là phần đã về đúng chỗ, ô vàng là vị trí vừa đổi/chèn (chọn, chèn), ô nhạt ở phần Nhanh là đoạn đang không xử lý. Sắp giảm dần thì các bước giống nhau nhưng đảo dấu so sánh (hộp vàng ở phần C); kết quả cuối ghi sẵn ở từng dãy.

Dãy 1: 9 1 2 10 8 5 7 4 6

Kết quả tăng dần: `1 2 4 5 6 7 8 9 10` · giảm dần: `10 9 8 7 6 5 4 2 1`

Chọn

đầu	9	1	2	10	8	5	7	4	6
i=0	**1**	**9**	2	10	8	5	7	4	6
i=1	1	**2**	**9**	10	8	5	7	4	6
i=2	1	2	**4**	10	8	5	7	**9**	6
i=3	1	2	4	**5**	8	**10**	7	9	6
i=4	1	2	4	5	**6**	10	7	9	**8**
i=5	1	2	4	5	6	**7**	**10**	9	8
i=6	1	2	4	5	6	7	**8**	9	**10**
i=7	1	2	4	5	6	7	8	9	10

đầu	
i=0	min = 1, đổi vị trí 0 ↔ 1
i=1	min = 2, đổi vị trí 1 ↔ 2
i=2	min = 4, đổi vị trí 2 ↔ 7
i=3	min = 5, đổi vị trí 3 ↔ 5
i=4	min = 6, đổi vị trí 4 ↔ 8
i=5	min = 7, đổi vị trí 5 ↔ 6
i=6	min = 8, đổi vị trí 6 ↔ 8
i=7	min = 9, đã đúng chỗ

Chèn

đầu	9	1	2	10	8	5	7	4	6
i=1	**1**	9	2	10	8	5	7	4	6
i=2	1	**2**	9	10	8	5	7	4	6
i=3	1	2	9	**10**	8	5	7	4	6
i=4	1	2	**8**	9	10	5	7	4	6
i=5	1	2	**5**	8	9	10	7	4	6
i=6	1	2	5	**7**	8	9	10	4	6
i=7	1	2	**4**	5	7	8	9	10	6
i=8	1	2	4	5	**6**	7	8	9	10

đầu	
i=1	chèn 1 vào vị trí 0
i=2	chèn 2 vào vị trí 1
i=3	chèn 10 vào vị trí 3
i=4	chèn 8 vào vị trí 2
i=5	chèn 5 vào vị trí 2
i=6	chèn 7 vào vị trí 3
i=7	chèn 4 vào vị trí 2
i=8	chèn 6 vào vị trí 4

Nổi bọt

đầu	9	1	2	10	8	5	7	4	6
lượt 1	1	2	9	8	5	7	4	6	10
lượt 2	1	2	8	5	7	4	6	9	10
lượt 3	1	2	5	7	4	6	8	9	10
lượt 4	1	2	5	4	6	7	8	9	10
lượt 5	1	2	4	5	6	7	8	9	10
lượt 6	1	2	4	5	6	7	8	9	10

đầu	
lượt 1	7 lần đổi chỗ
lượt 2	5 lần đổi chỗ
lượt 3	4 lần đổi chỗ
lượt 4	2 lần đổi chỗ
lượt 5	1 lần đổi chỗ
lượt 6	0 lần đổi chỗ, đã xong (các lượt sau không đổi)

Trộn

Chia đôi (giua = (dau + cuoi) / 2)

```
[9 1 2 10 8 5 7 4 6]
  [9 1 2 10 8]
    [9 1 2]
      [9 1]
        [9]
        [1]
      [2]
    [10 8]
      [10]
      [8]
  [5 7 4 6]
    [5 7]
      [5]
      [7]
    [4 6]
      [4]
      [6]
```

Trộn từ dưới lên (theo thứ tự thực hiện)

```
[9] + [1] → [1 9]
[1 9] + [2] → [1 2 9]
[10] + [8] → [8 10]
[1 2 9] + [8 10] → [1 2 8 9 10]
[5] + [7] → [5 7]
[4] + [6] → [4 6]
[5 7] + [4 6] → [4 5 6 7]
[1 2 8 9 10] + [4 5 6 7] → [1 2 4 5 6 7 8 9 10]
```

Nhanh (pivot = phần tử giữa)

đầu	9	1	2	10	8	5	7	4	6
[0..8]	6	1	2	4	7	5	8	10	9
[0..5]	2	1	6	4	7	5	8	10	9
[0..1]	1	2	6	4	7	5	8	10	9
[2..5]	1	2	4	6	7	5	8	10	9
[3..5]	1	2	4	6	5	7	8	10	9
[3..4]	1	2	4	5	6	7	8	10	9
[6..8]	1	2	4	5	6	7	8	9	10
[6..7]	1	2	4	5	6	7	8	9	10

pivot x = 8; sau chia: trái [0..5], phải [6..8]

pivot x = 2; sau chia: trái [0..1], phải [2..5]

pivot x = 2; sau chia: trái [0..0], phải [1..1]

pivot x = 4; sau chia: trái [2..2], phải [3..5]

pivot x = 7; sau chia: trái [3..4], phải [5..5]

pivot x = 6; sau chia: trái [3..3], phải [4..4]

pivot x = 10; sau chia: trái [6..7], phải [8..8]

pivot x = 8; sau chia: trái [6..5], phải [7..7]

Dãy 2: 5 15 12 2 10 12 9 1 9 3 2 3

Kết quả tăng dần: `1 2 2 3 3 5 9 9 10 12 12 15` · giảm dần: `15 12 12 10 9 9 5 3 3 2 2 1`

Chọn

đầu	5	15	12	2	10	12	9	1	9	3	2	3
i=0	**1**	15	12	2	10	12	9	**5**	9	3	2	3
i=1	1	**2**	12	**15**	10	12	9	5	9	3	2	3
i=2	1	2	**2**	15	10	12	9	5	9	3	**12**	3
i=3	1	2	2	**3**	10	12	9	5	9	**15**	12	3
i=4	1	2	2	3	**3**	12	9	5	9	15	12	**10**
i=5	1	2	2	3	3	**5**	9	**12**	9	15	12	10
i=6	1	2	2	3	3	5	9	12	9	15	12	10
i=7	1	2	2	3	3	5	9	**9**	**12**	15	12	10
i=8	1	2	2	3	3	5	9	9	**10**	15	12	**12**
i=9	1	2	2	3	3	5	9	9	10	**12**	**15**	12
i=10	1	2	2	3	3	5	9	9	10	12	**12**	**15**

min = 1, đổi vị trí 0 ↔ 7

min = 2, đổi vị trí 1 ↔ 3

min = 2, đổi vị trí 2 ↔ 10

min = 3, đổi vị trí 3 ↔ 9

min = 3, đổi vị trí 4 ↔ 11

min = 5, đổi vị trí 5 ↔ 7

min = 9, đã đúng chỗ

min = 9, đổi vị trí 7 ↔ 8

min = 10, đổi vị trí 8 ↔ 11

min = 12, đổi vị trí 9 ↔ 10

min = 12, đổi vị trí 10 ↔ 11

Chèn

đầu	5	15	12	2	10	12	9	1	9	3	2	3	
i=1	5	**15**	12	2	10	12	9	1	9	3	2	3	chèn 15 vào vị trí 1
i=2	5	**12**	15	2	10	12	9	1	9	3	2	3	chèn 12 vào vị trí 1
i=3	**2**	5	12	15	10	12	9	1	9	3	2	3	chèn 2 vào vị trí 0
i=4	2	5	**10**	12	15	12	9	1	9	3	2	3	chèn 10 vào vị trí 2
i=5	2	5	10	12	**12**	15	9	1	9	3	2	3	chèn 12 vào vị trí 4
i=6	2	5	**9**	10	12	12	15	1	9	3	2	3	chèn 9 vào vị trí 2
i=7	**1**	2	5	9	10	12	12	15	9	3	2	3	chèn 1 vào vị trí 0
i=8	1	2	5	9	**9**	10	12	12	15	3	2	3	chèn 9 vào vị trí 4
i=9	1	2	**3**	5	9	9	10	12	12	15	2	3	chèn 3 vào vị trí 2
i=10	1	2	**2**	3	5	9	9	10	12	12	15	3	chèn 2 vào vị trí 2
i=11	1	2	2	3	**3**	5	9	9	10	12	12	15	chèn 3 vào vị trí 4

Nổi bọt

đầu	5	15	12	2	10	12	9	1	9	3	2	3	
lượt 1	5	12	2	10	12	9	1	9	3	2	3	15	10 lần đổi chỗ
lượt 2	5	2	10	12	9	1	9	3	2	3	12	15	8 lần đổi chỗ
lượt 3	2	5	10	9	1	9	3	2	3	12	12	15	7 lần đổi chỗ
lượt 4	2	5	9	1	9	3	2	3	10	12	12	15	6 lần đổi chỗ
lượt 5	2	5	1	9	3	2	3	9	10	12	12	15	4 lần đổi chỗ
lượt 6	2	1	5	3	2	3	9	9	10	12	12	15	4 lần đổi chỗ
lượt 7	1	2	3	2	3	5	9	9	10	12	12	15	4 lần đổi chỗ
lượt 8	1	2	2	3	3	5	9	9	10	12	12	15	1 lần đổi chỗ
lượt 9	1	2	2	3	3	5	9	9	10	12	12	15	0 lần đổi chỗ, đã xong (các lượt sau không đổi)

Trộn

Chia đôi (giua = (dau + cuoi) / 2)

```
[5 15 12 2 10 12 9 1 9 3 2 3]
  [5 15 12 2 10 12]
    [5 15 12]
      [5 15]
        [5]
        [15]
      [12]
    [2 10 12]
      [2 10]
        [2]
        [10]
      [12]
  [9 1 9 3 2 3]
    [9 1 9]
      [9 1]
        [9]
        [1]
      [9]
    [3 2 3]
      [3 2]
        [3]
        [2]
      [3]
```

Trộn từ dưới lên (theo thứ tự thực hiện)

```
[5] + [15] → [5 15]
[5 15] + [12] → [5 12 15]
[2] + [10] → [2 10]
[2 10] + [12] → [2 10 12]
[5 12 15] + [2 10 12] → [2 5 10 12 12 15]
[9] + [1] → [1 9]
[1 9] + [9] → [1 9 9]
[3] + [2] → [2 3]
[2 3] + [3] → [2 3 3]
[1 9 9] + [2 3 3] → [1 2 3 3 9 9]
[2 5 10 12 12 15] + [1 2 3 3 9 9] → [1 2 2 3 3 5 9 9 10 12 12 15]
```

Nhanh (pivot = phần tử giữa)

đầu	5	15	12	2	10	12	9	1	9	3	2	3
[0..11]	5	3	2	2	10	3	9	1	9	12	12	15
[0..8]	5	3	2	2	9	3	9	1	10	12	12	15
[0..7]	1	2	2	3	9	3	9	5	10	12	12	15
[0..1]	1	2	2	3	9	3	9	5	10	12	12	15
[3..7]	1	2	2	3	9	3	9	5	10	12	12	15
[4..7]	1	2	2	3	3	9	9	5	10	12	12	15
[5..7]	1	2	2	3	3	5	9	9	10	12	12	15
[9..11]	1	2	2	3	3	5	9	9	10	12	12	15
[10..11]	1	2	2	3	3	5	9	9	10	12	12	15

pivot x = 12; sau chia: trái [0..8], phải [9..11]

pivot x = 10; sau chia: trái [0..7], phải [8..8]

pivot x = 2; sau chia: trái [0..1], phải [3..7]

pivot x = 1; sau chia: trái [0..-1], phải [1..1]

pivot x = 3; sau chia: trái [3..3], phải [4..7]

pivot x = 3; sau chia: trái [4..4], phải [5..7]

pivot x = 9; sau chia: trái [5..5], phải [7..7]

pivot x = 12; sau chia: trái [9..9], phải [10..11]

pivot x = 12; sau chia: trái [10..9], phải [11..11]

Dãy 3: 51 61 21 12 101 121 91 101 91 31

Kết quả tăng dần: `12 21 31 51 61 91 91 101 101 121` · giảm dần: `121 101 101 91 91 61 51 31 21 12`

Chọn

đầu	51	61	21	12	101	121	91	101	91	31	
i=0	**12**	61	21	**51**	101	121	91	101	91	31	min = 12, đổi vị trí 0 ↔ 3
i=1	12	**21**	**61**	51	101	121	91	101	91	31	min = 21, đổi vị trí 1 ↔ 2
i=2	12	21	**31**	51	101	121	91	101	91	**61**	min = 31, đổi vị trí 2 ↔ 9
i=3	12	21	31	51	101	121	91	101	91	61	min = 51, đã đúng chỗ
i=4	12	21	31	51	**61**	121	91	101	91	**101**	min = 61, đổi vị trí 4 ↔ 9
i=5	12	21	31	51	61	**91**	**121**	101	91	101	min = 91, đổi vị trí 5 ↔ 6
i=6	12	21	31	51	61	91	**91**	101	**121**	101	min = 91, đổi vị trí 6 ↔ 8
i=7	12	21	31	51	61	91	91	101	121	101	min = 101, đã đúng chỗ
i=8	12	21	31	51	61	91	91	101	**101**	**121**	min = 101, đổi vị trí 8 ↔ 9

Chèn

đầu	51	61	21	12	101	121	91	101	91	31	
i=1	51	**61**	21	12	101	121	91	101	91	31	chèn 61 vào vị trí 1
i=2	**21**	51	61	12	101	121	91	101	91	31	chèn 21 vào vị trí 0
i=3	**12**	21	51	61	101	121	91	101	91	31	chèn 12 vào vị trí 0
i=4	12	21	51	61	**101**	121	91	101	91	31	chèn 101 vào vị trí 4
i=5	12	21	51	61	101	**121**	91	101	91	31	chèn 121 vào vị trí 5
i=6	12	21	51	61	**91**	101	121	101	91	31	chèn 91 vào vị trí 4
i=7	12	21	51	61	91	101	**101**	121	91	31	chèn 101 vào vị trí 6
i=8	12	21	51	61	91	**91**	101	101	121	31	chèn 91 vào vị trí 5
i=9	12	21	**31**	51	61	91	91	101	101	121	chèn 31 vào vị trí 2

Nổi bọt

đầu	51	61	21	12	101	121	91	101	91	31	
lượt 1	51	21	12	61	101	91	101	91	31	121	6 lần đổi chỗ
lượt 2	21	12	51	61	91	101	91	31	101	121	5 lần đổi chỗ
lượt 3	12	21	51	61	91	91	31	101	101	121	3 lần đổi chỗ
lượt 4	12	21	51	61	91	31	91	101	101	121	1 lần đổi chỗ
lượt 5	12	21	51	61	31	91	91	101	101	121	1 lần đổi chỗ
lượt 6	12	21	51	31	61	91	91	101	101	121	1 lần đổi chỗ
lượt 7	12	21	31	51	61	91	91	101	101	121	1 lần đổi chỗ
lượt 8	12	21	31	51	61	91	91	101	101	121	0 lần đổi chỗ, đã xong (các lượt sau không đổi)

Trộn

Chia đôi (giua = (dau + cuoi) / 2)

```
[51 61 21 12 101 121 91 101 91 31]
  [51 61 21 12 101]
    [51 61 21]
      [51 61]
        [51]
        [61]
      [21]
    [12 101]
      [12]
      [101]
  [121 91 101 91 31]
    [121 91 101]
      [121 91]
        [121]
        [91]
      [101]
    [91 31]
      [91]
      [31]
```

Trộn từ dưới lên (theo thứ tự thực hiện)

```
[51] + [61] → [51 61]
[51 61] + [21] → [21 51 61]
[12] + [101] → [12 101]
[21 51 61] + [12 101] → [12 21 51 61 101]
[121] + [91] → [91 121]
[91 121] + [101] → [91 101 121]
[91] + [31] → [31 91]
[91 101 121] + [31 91] → [31 91 91 101 121]
[12 21 51 61 101] + [31 91 91 101 121] → [12 21 31 51 61 91 91 101 101 121]
```

Nhanh (pivot = phần tử giữa)

đầu	51	61	21	12	101	121	91	101	91	31	
[0..9]	51	61	21	12	31	91	91	101	121	101	pivot x = 101; sau chia: trái [0..6], phải [8..9]
[0..6]	12	61	21	51	31	91	91	101	121	101	pivot x = 12; sau chia: trái [0..0], phải [1..6]
[1..6]	12	31	21	51	61	91	91	101	121	101	pivot x = 51; sau chia: trái [1..2], phải [4..6]
[1..2]	12	21	31	51	61	91	91	101	121	101	pivot x = 31; sau chia: trái [1..1], phải [2..2]
[4..6]	12	21	31	51	61	91	91	101	121	101	pivot x = 91; sau chia: trái [4..5], phải [6..6]
[4..5]	12	21	31	51	61	91	91	101	121	101	pivot x = 61; sau chia: trái [4..3], phải [5..5]
[8..9]	12	21	31	51	61	91	91	101	101	121	pivot x = 121; sau chia: trái [8..8], phải [9..9]